6: Diziler (Arrays)



Problem

adet klavyeden okuyan, bu hesaplayan ve adet bulan

.

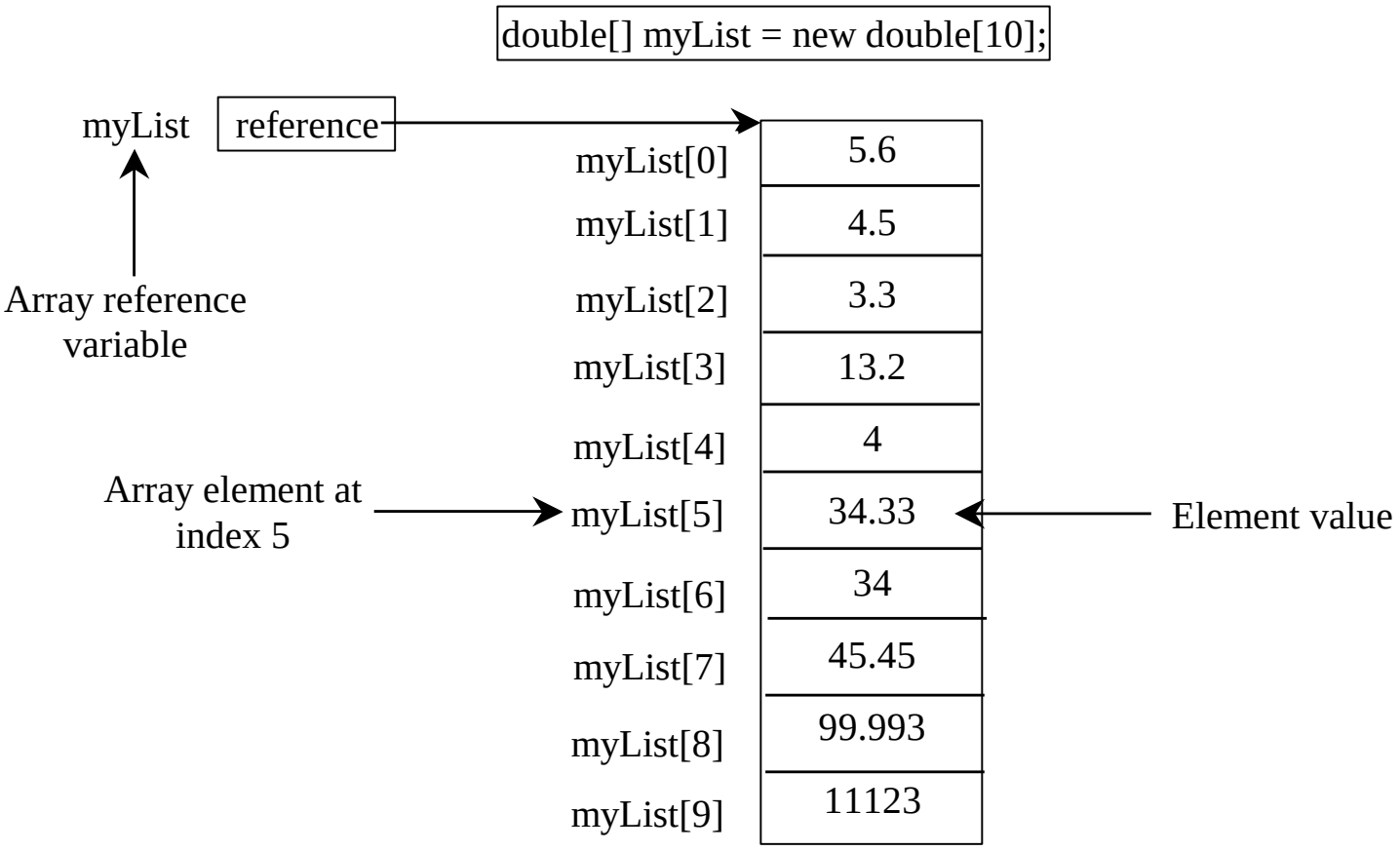


AnalyzeNumbers

Run

Run with prepared input




double[] myList = new double[10];
myList
reference
Array reference variable
Array element at index 5
myList[0]
myList[1]
myList[2]
myList[3]
myList[4]
myList[5]
myList[6]
myList[7]
myList[8]
myList[9]
5.6
4.5
3.3
13.2
4
34.33
34
45.45
99.993
11123
Element value

- `datatype[] arrayRefVar;`

  :

  `double[] myList;`

- `datatype arrayRefVar[];` // Bu stile izin verilir, ancak tercih edilmez

  :

  `double myList[];`



```
arrayRefVar = new datatype[arraySize];

    :
myList = new double[10];


myList[0]
myList[9]
```



- ```
  datatype[] arrayRefVar = new
     datatype[arraySize];
  ```

  ```
  double[] myList = new double[10];
  ```

- ```
  datatype arrayRefVar[] = new
    datatype[arraySize];
  ```

  ```
  double myList[] = new double[10];
  ```



Bir dizi sonra, boyutu sabittir, d . Dizinin boyutu belirtilen kod ile bulunabilir.

arrayRefVar.length

,

myList.length returns 10



D

Bir dizi

gibidir.

(Numerik) ilkel veri tipleri 0,

Char veri tipi '\u0000'

Boolean veri tipi ise ***false*** .



Indexed

Dizi indeks . Dizi indeksleri 0 yani 0 dan arrayRefVar.length-1'e kadar devam eder. 6.1'deki myList on adet double ve indeksler 0 - 9 .

Dizideki her bir olarak bilinen kullanarak temsil edilir:

```
arrayRefVar[index];
```



- Bir dizi sonra, bir normal bir . kod myList [0] ve myList [1] 'deki toplayarak myList [2]' ye atar.

```
myList[2] = myList[0] + myList[1];
```



Initializers)

- diziyi bildirme,

- `double[] myList = {1.9, 2.9, 3.4, 3.5};`



```
double[] myList = {1.9, 2.9, 3.4, 3.5};
```

```
double[] myList = new double[4];
myList[0] = 1.9;
myList[1] = 2.9;
myList[2] = 3.4;
myList[3] = 3.5;
```



UYARI !

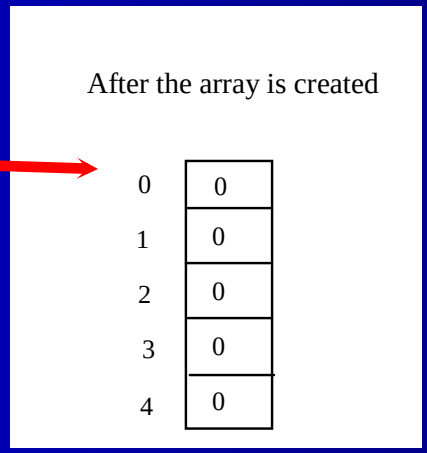
kullanarak, dizinin bir ifadede bildirmeniz, ve gerekir. bir neden olur. :

```
double[] myList;

    myList = {1.9, 2.9, 3.4, 3.5};
```



```
public class Test {
  public static void main(String[] args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```



```
public class Test {
  public static void main(String[] args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

After the array is created

0	0
1	0
2	0
3	0
4	0

Dizilerle

i (=1)

```
public class Test {
  public static void ma (String[] args) {
    int[] values = nev int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

After the array is created

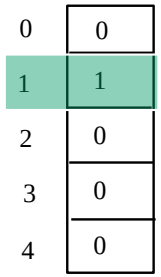
0	0
1	0
2	0
3	0
4	0



, value[1]
1

```
public class Test {
  public static void main(Strin      gs) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

After the first iteration



```
public class Test {
  public static void main(String[] args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

After the first iteration

0	0
1	1
2	0
3	0
4	0



```
public class Test {
  public static void main(String[]
      args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] +
      values[4];
  }
}
```

i (= 2)

After the first iteration

0	0
1	1
2	0
3	0
4	0



values[2] = 2 + values[2-1]

```
public class Test {
  public static void main(Strin      gs) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

After the second iteration

0	0
1	1
2	3
3	0
4	0



```
public class Test {
  public static void main(String[] args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

After the second iteration

0	0
1	1
2	3
3	0
4	0



i (= 3)

```
public class Test {
  public static void main(String[] args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

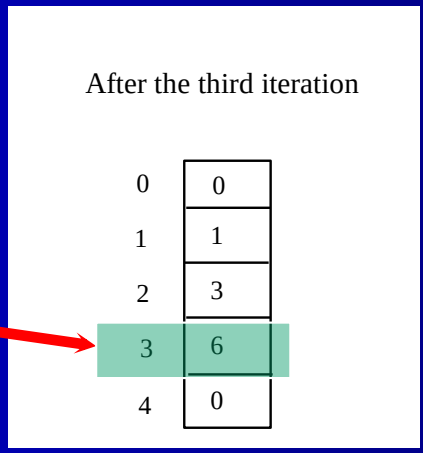
After the second iteration

0	0
1	1
2	3
3	0
4	0



, values[3] = 6 (3 + 3)

```
public class Test {
  public static void main(String[] [illegible] {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```



```
public class Test {
  public static void main(String[] args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

After the third iteration

0	0
1	1
2	3
3	6
4	0



i (= 4)

```
public class Test {
  public static void main(String[] args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

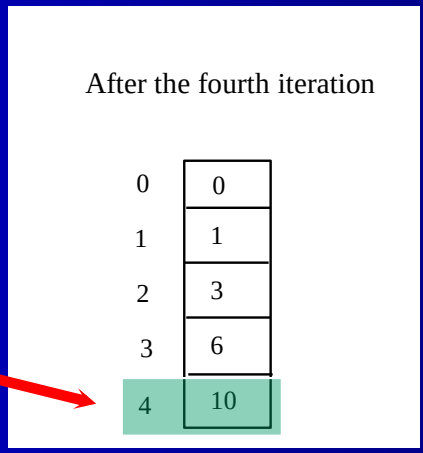
After the third iteration

0	0
1	1
2	3
3	6
4	0



, values[4] = 10 (4 + 6)

```
public class Test {
  public static void main(Str      gs) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++)
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```



```
public class Test {
  public static void main(String[] arg
          {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

After the fourth iteration

0	0
1	1
2	3
3	6
4	10

i (= 5) < 5

```
public class Test {
  public static void main(String[] args) {
    int[] values = new int[5];
    for (int i = 1; i < 5; i++) {
      values[i] = i + values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```

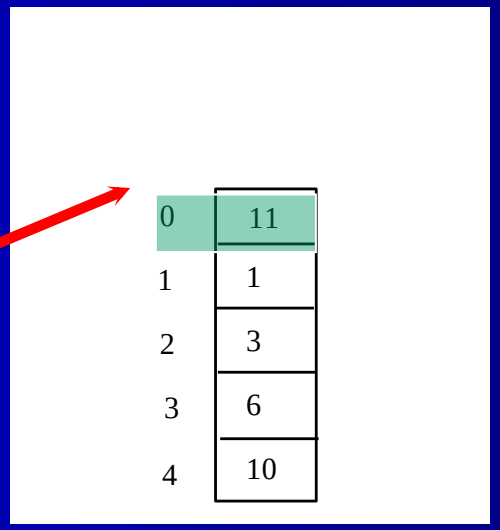
After the fourth iteration

0	0
1	1
2	3
3	6
4	10



, values[0] = 11 (1 + 10)

```
public class Test {
  public static void ma                 [] args) {
    int[] values = new
    for (int i = 1; i <                 ++) {
      values[i] =  i                  values[i-1];
    }
    values[0] = values[1] + values[4];
  }
}
```





Girdi D Sahip Dizileri B

```
java.util.Scanner input = new java.util.Scanner(System.in);
System.out.print("Enter " + myList.length + " values: ");
for (int i = 0; i < myList.length; i++)
  myList[i] = input.nextDouble();
```

Dizileri Rasgele D B

```
for (int i = 0; i < myList.length; i++) {
  myList[i] = Math.random() * 100;
}
```



Dizileri Y

```
for (int i = 0; i < myList.length; i++) {
  System.out.print(myList[i] + " ");
}
```



Toplama

```
double total = 0;
for (int i = 0; i < myList.length; i++) {
  total += myList[i];
}
```



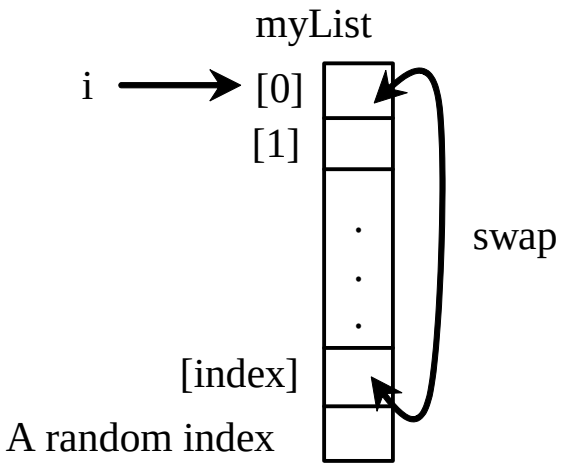
Dizinin En B E Bulma

```
double max = myList[0];
for (int i = 1; i < myList.length; i++) {
  if (myList[i] > max) max = myList[i];
}
```



```
for (int i = 0; i < myList.length; i++) {
  // Generate an index j randomly
  int index = (int)(Math.random()
    * myList.length);

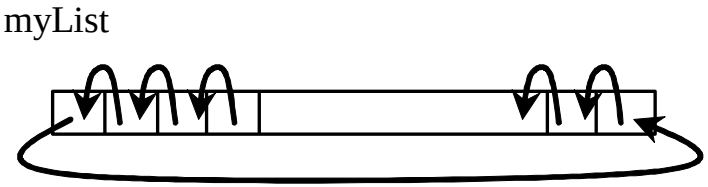
  // Swap myList[i] with myList[j]
  double temp = myList[i];
  myList[i] = myList[index];
  myList[index] = temp;
}
```



```
double temp = myList[0]; // Retain the first element

// Shift elements left
for (int i = 1; i < myList.length; i++) {
  myList[i - 1] = myList[i];
}

// Move the first element to fill in the last position
myList[myList.length - 1] = temp;
```





)

JDK 1.5 Bir indeks kullanmadan diziyi olarak yeni bir for . kod myList dizisindeki .

```
for (double value: myList)
  System.out.println(value);
```

Genel olarak, syntax

```
for (elementType value: arrayRefVar) {
  // Process the value
}
```



Diyelim ki, 10 numara isimli oyununu oynuyorsunuz. Her bilette 1 ile 99 10 benzersiz numara var. bilet . 1'den 99'a kadar olan kapsayacak biletlerinizi almak istediniz. Bir dosyadaki bilet okuyan ve olup kontrol eden bir program . Dosyadaki son 0 .

Lotto Numbers Sample Data

LottoNumbers

Run

Problem: skambil Destesi

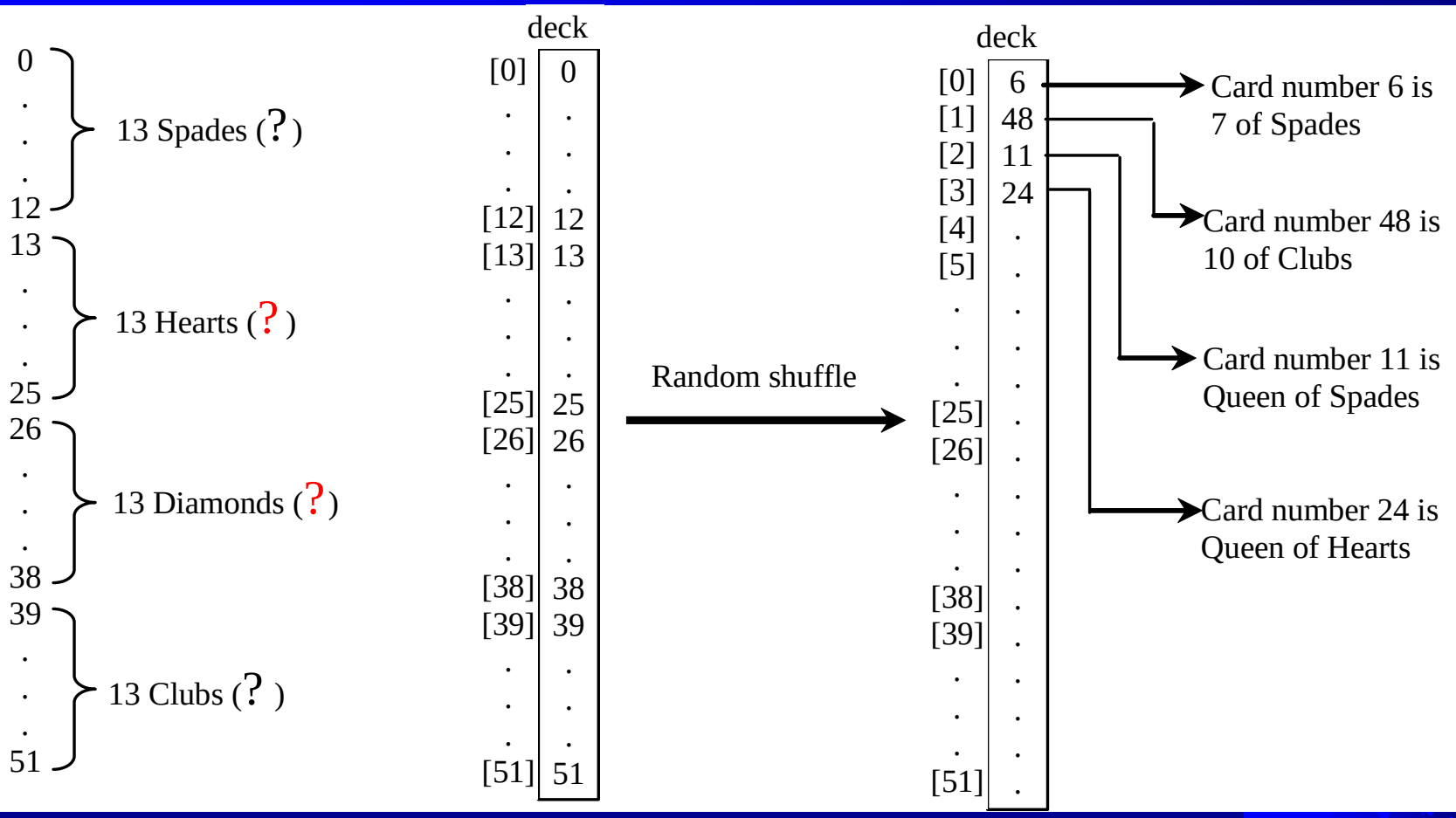
52 lik kart destesinden rastgele kart alan bir program
. kartlar gibi 0 ila 52
ile dolu, deck verilen bir dizi
:

```
int[] deck = new int[52];
// Initialize cards
for (int i = 0; i < deck.length; i++)
  deck[i] = i;
```

DeckOfCards

Run

Problem: skambil Destesi



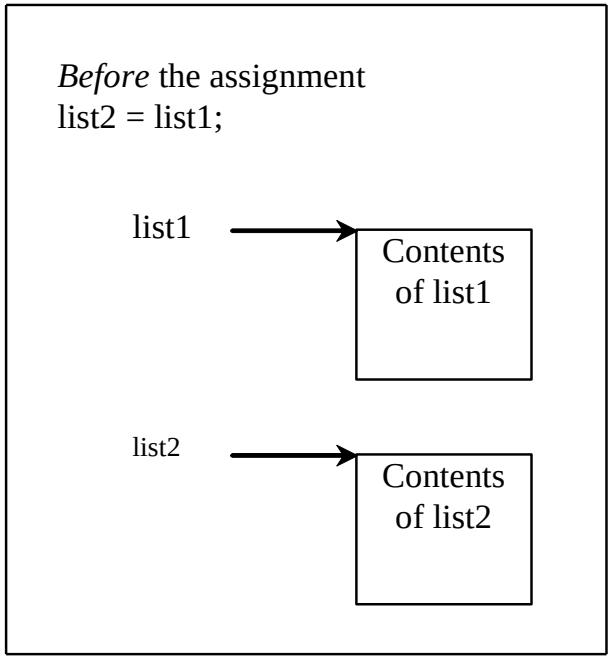
GUI Demo (picking four cards)

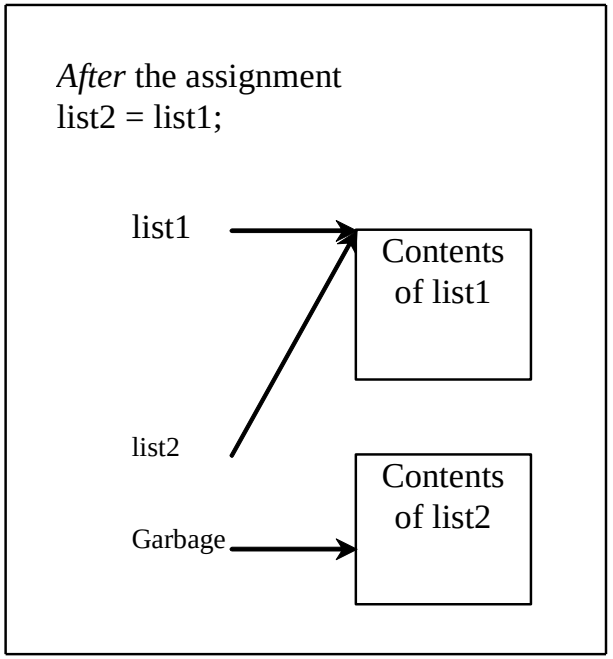
DeckOfCards

Run

gerekebilir. Bu gibi durumlarda, atama deyimini (=)

list2 = list1;





```
int[] sourceArray = {2, 3, 1, 5, 10};
int[] targetArray = new int[sourceArray.length];

for (int i = 0; i < sourceArray.length; i++)
   targetArray[i] = sourceArray[i];
```



arraycopy

```
arraycopy(sourceArray, src_pos,
  targetArray, tar_pos, length);

    :

System.arraycopy(sourceArray, 0,
  targetArray, 0, sourceArray.length);
```



```
public static void printArray(int[] array) {
  for (int i = 0; i < array.length; i++) {
    System.out.print(array[i] + " ");
  }
}
```

```
//Metodu çağır.

int[] list = {3, 1, 2, 6, 4, 2};
printArray(list);
```

```
Invoke the method
printArray(new int[]{3, 1, 2, 6, 4, 2});
```

Anonim Dizi



Anonim Dizi

printArray(new int[]{3, 1, 2, 6, 4, 2});

arak bir dizi

:new dataType[]{literal0, literal1, ..., literalk};



Pass By Value

Java, bir metoda parametreleri iletmek pass by value
. veri tiplerinin
iletmek ile dizileri iletmek .

bir parametresi iletilir.
yerel parametrenin
etkilemez.

Bir dizi bir parametresi parametrenin bir diziye referans ; bu referans metoda iletilir. Metot dizide
olarak orijinal diziyi etkiler.

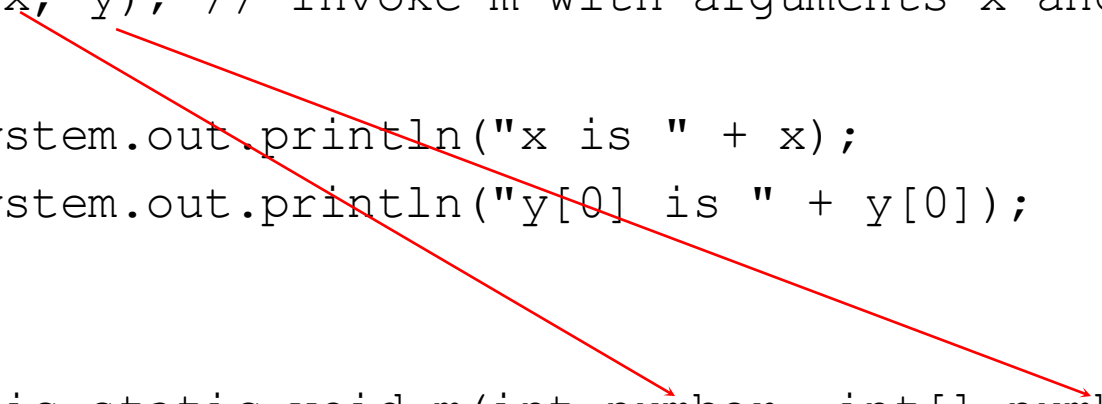


```
public class Test {
  public static void main(String[] args) {
    int x = 1; // x represents an int value
    int[] y = new int[10]; // y represents an array of int values

    m(x, y); // Invoke m with arguments x and y

    System.out.println("x is " + x);
    System.out.println("y[0] is " + y[0]);
  }

  public static void m(int number, int[] numbers) {
    number = 1001; // Assign a new value to number
    numbers[0] = 5555; // Assign a new value to numbers[0]
  }
}
```



Dizileri Parametre Olarak



iletmenin

.

TestPassArray

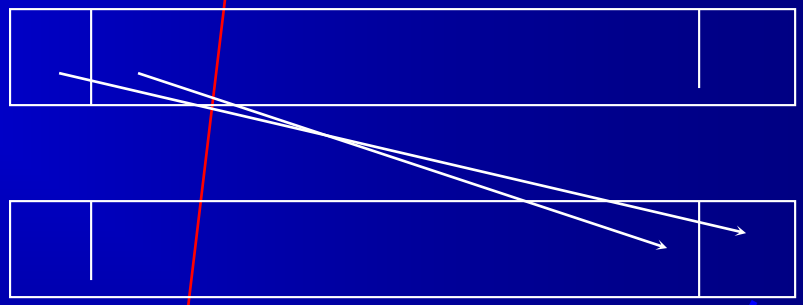
Run

Bir Metottan

```
public static int[] reverse(int[] list) {
  int[] result = new int[list.length];

  for (int i = 0, j = result.length - 1;
       i < list.length; i++, j--) {
    result[j] = list[i];
  }

  return result;
}
```



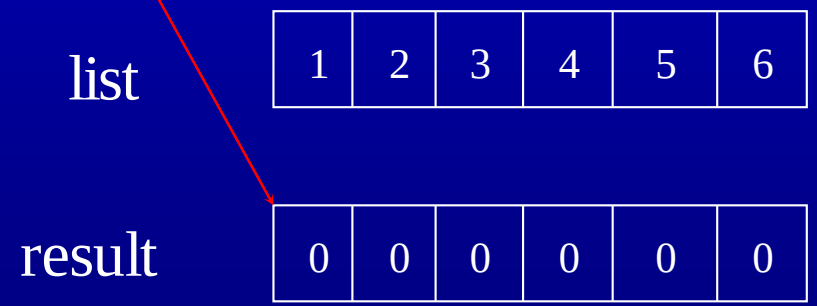

```
int[] list1 = new int[]{1, 2, 3, 4, 5, 6};
int[] list2 = reverse(list1);
```

```
int[] list1 = {1, 2, 3, 4, 5, 6};
int[] list2 = reverse(list1);
```

```
public static int[] reverse(int[] list) {
  int[] result = new int[list.length];

  for (int i = 0, j = result.length - 1;
       i < list.length; i++, j--) {
    result[j] = list[i];
  }

  return result;
}
```



```
int[] list1 = new int[]{1, 2, 3, 4, 5, 6};
int[] list2 = reverse(list1);
```

i = 0 ve j = 5

```
public static int[] reverse(int[] list) {
  int[] result = new int[list.length];

  for (int i = 0, j = result.length - 1;
       i < list.length; i++, j--) {
    result[j] = list[i];
  }

  return result;
}
```



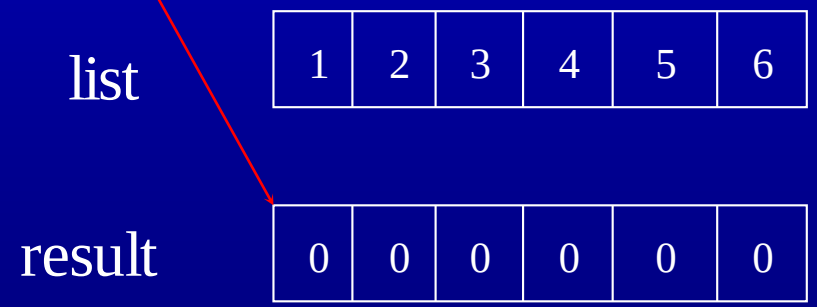
```
int[] list1 = new int[]{1, 2, 3, 4, 5, 6};
int[] list2 = reverse(list1);
```

```
public static int[] reverse(int[] list) {
  int[] result = new int[list.length];

  for (int i = 0, j = result.length - 1;
       i < list.length; i++, j--) {
    result[j] = list[i];
  }

  return result;
}
```

i (= 0)



```
int[] list1 = new int[]{1, 2, 3, 4, 5, 6};
int[] list2 = reverse(list1);
```

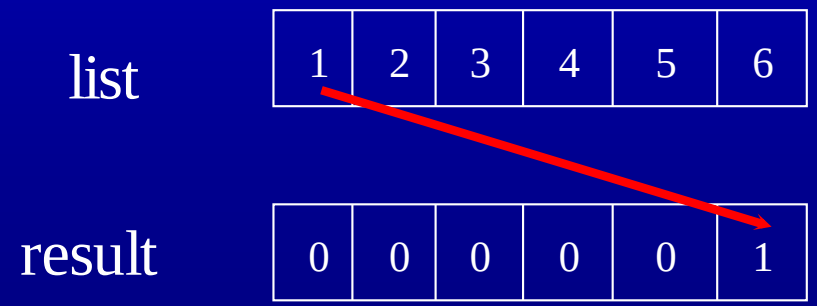
```
public static int[] reverse(int[] list) {
  int[] result = new int[list.length];

  for (int i = 0, j = result.length - 1;
       i < list.length; i++, j--) {
    result[j] = list[i];
  }

  return result;
}
```

i = 0 ve j = 5

list[0] result[5]





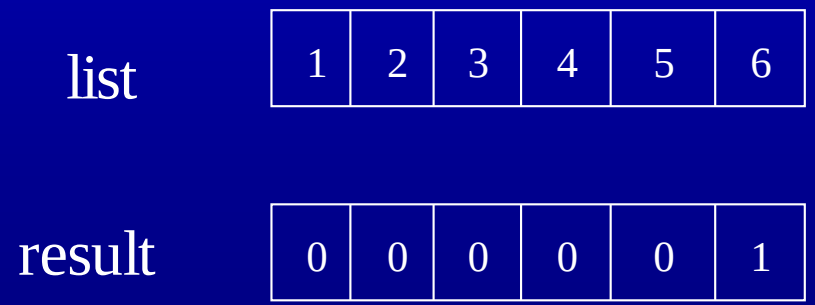
```
int[] list1 = new int[]{1, 2, 3, 4, 5, 6};
int[] list2 = reverse(list1);
```

```
public static int[] reverse(int[] list) {
  int[] result = new int[list.length];

  for (int i = 0, j = result.length - 1;
       i < list.length; i++, j--) {
    result[j] = list[i];
  }

  return result;
}
```

, i 1
j 4 olur.



```
int[] list1 = new int[]{1, 2, 3, 4, 5, 6};
int[] list2 = reverse(list1);
```

```
public static int[] reverse(int[] list) {
  int[] result = new int[list.length];

  for (int i = 0, j = result.length - 1;
       i < list.length; i++, j--) {
    result[j] = list[i];
  }

  return result;
}
```

i (=1

list

1	2	3	4	5	6

result

0	0	0	0	0	1



```
int[] list1 = new int[]{1, 2, 3, 4, 5, 6};
int[] list2 = reverse(list1);
```

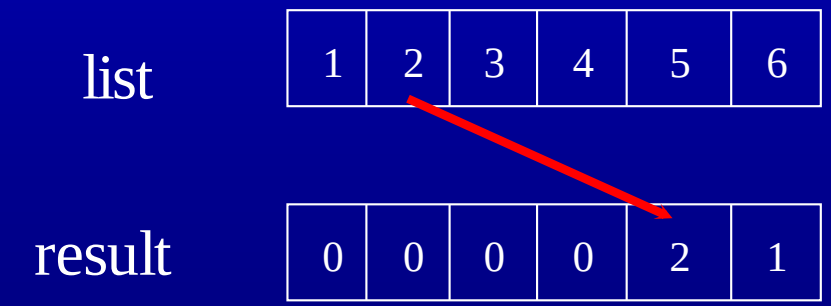
```
public static int[] reverse(int[] list) {
  int[] result = new int[list.length];

  for (int i = 0, j = result.length -
       i < list.length; i++, j--) {
    result[j] = list[i];
  }

  return result;
}
```

i = 1 ve j = 4

list[1] result[4]



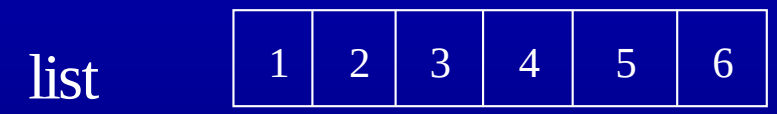
```
int[] list1 = new int[]{1, 2, 3, 4, 5, 6};
int[] list2 = reverse(list1);
```

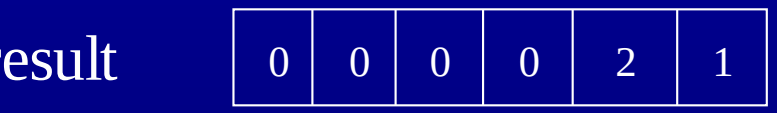
```
public static int[] reverse(int[] list) {
  int[] result = new int[list.length];

  for (int i = 0, j = result.length - 1;
       i < list.length; i++, j--) {
    result[j] = list[i];
  }

  return result;
}
```

, i 2
j 3 olur.







```
int[] list1 = new int[]{1, 2, 3, 4, 5, 6};
int[] list2 = reverse(list1);
```

```
public static int[] reverse(int[] list) {
  int[] result = new int[list.length];

  for (int i = 0, j = result.length - 1;
       i < list.length; i++, j--) {
    result[j] = list[i];
  }

  return result;
}
```

i (=2)

list

1	2	3	4	5	6

result

0	0	0	0	2	1



```
int[] list1 = new int[]{1, 2, 3, 4, 5, 6};
int[] list2 = reverse(list1);
```

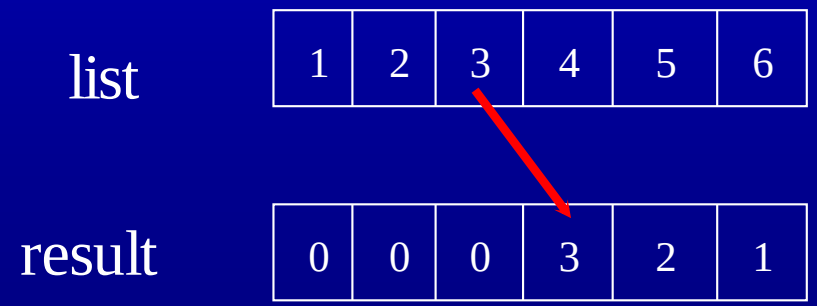
```
public static int[] reverse(int[] list) {
  int[] result = new int[list.length];

  for (int i = 0, j = result.length - 1;
       i < list.length; i++, j--) {
    result[j] = list[i];
  }

  return result;
}
```

i = 2 ve j = 3

list[2] result[3]



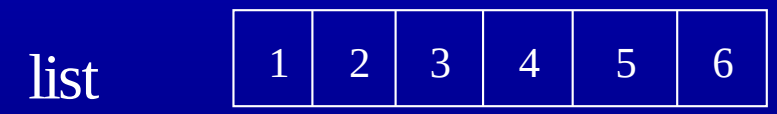
```
int[] list1 = new int[]{1, 2, 3, 4, 5, 6};
int[] list2 = reverse(list1);
```

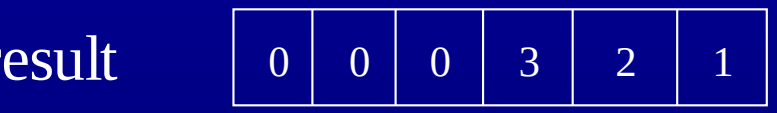
```
public static int[] reverse(int[] list) {
  int[] result = new int[list.length];

  for (int i = 0, j = result.length - 1;
       i < list.length; i++, j--) {
    result[j] = list[i];
  }

  return result;
}
```

, i 3
j 2 olur.







```
int[] list1 = new int[]{1, 2, 3, 4, 5, 6};
int[] list2 = reverse(list1);
```

```
public static int[] reverse(int[] list) {
  int[] result = new int[list.length];

  for (int i = 0, j = result.length - 1;
       i < list.length; i++, j--) {
    result[j] = list[i];
  }

  return result;
}
```

i (=3)

list	1	2	3	4	5	6

result	0	0	0	3	2	1



```
int[] list1 = new int[]{1, 2, 3, 4, 5, 6};
int[] list2 = reverse(list1);
```

```
public static int[] reverse(int[] list) {
  int[] result = new int[list.length];

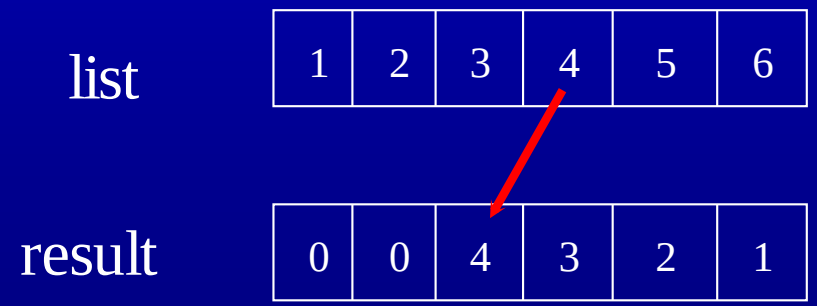
  for (int i = 0, j = result.length -
       i < list.length; i++, j--) {
    result[j] = list[i];
  }

  return result;
}
```

i = 3 ve j = 2

list[3] result[2]

]



```
int[] list1 = new int[]{1, 2, 3, 4, 5, 6};
int[] list2 = reverse(list1);
```

```
public static int[] reverse(int[] list) {
  int[] result = new int[list.length];

  for (int i = 0, j = result.length - 1;
       i < list.length; i++, j--) {
    result[j] = list[i];
  }

  return result;
}
```

, i 4
j 1 olur.

list

1	2	3	4	5	6

result

0	0	4	3	2	1



```
int[] list1 = new int[]{1, 2, 3, 4, 5, 6};
int[] list2 = reverse(list1);
```

```
public static int[] reverse(int[] list) {
  int[] result = new int[list.length];

  for (int i = 0, j = result.length - 1;
       i < list.length; i++, j--) {
    result[j] = list[i];
  }

  return result;
}
```

i (=4)

list

1	2	3	4	5	6

result

0	0	4	3	2	1



```
int[] list1 = new int[]{1, 2, 3, 4, 5, 6};
int[] list2 = reverse(list1);
```

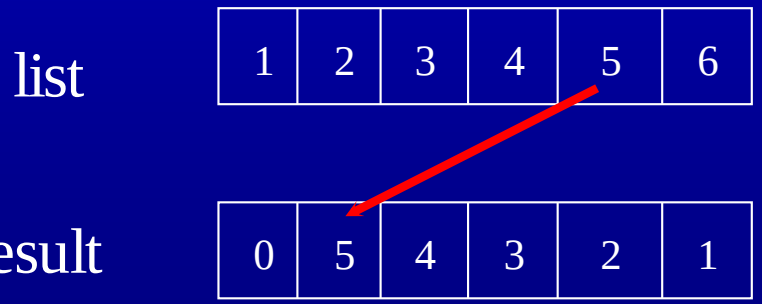
```
public static int[] reverse(int[] list) {
  int[] result = new int[list.length];

  for (int i = 0, j = result.length -
       i < list.length; i++, j--) {
    result[j] = list[i];
  }

  return result;
}
```

i = 4 ve j = 1

list[4] result[1]



```
int[] list1 = new int[]{1, 2, 3, 4, 5, 6};
int[] list2 = reverse(list1);
```

```
public static int[] reverse(int[] list) {
  int[] result = new int[list.length];

  for (int i = 0, j = result.length - 1;
       i < list.length; i++, j--) {
    result[j] = list[i];
  }

  return result;
}
```

, i 5
j 0 olur.



list

1	2	3	4	5	6

result

0	5	4	3	2	1

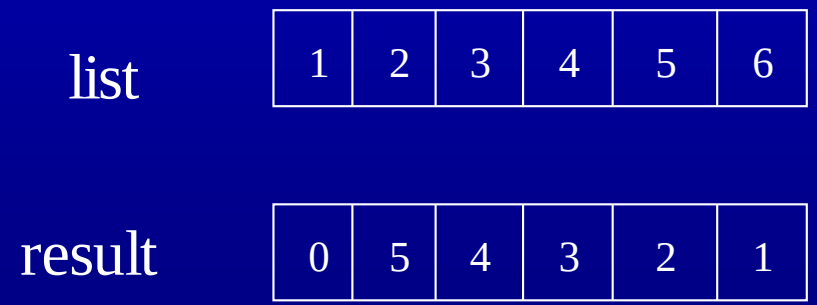
```
int[] list1 = new int[]{1, 2, 3, 4, 5, 6};
int[] list2 = reverse(list1);
```

```
public static int[] reverse(int[] list) {
  int[] result = new int[list.length];

  for (int i = 0, j = result.length - 1;
       i < list.length; i++, j--) {
    result[j] = list[i];
  }

  return result;
}
```

i (=5)



```
int[] list1 = new int[]{1, 2, 3, 4, 5, 6};
int[] list2 = reverse(list1);
```

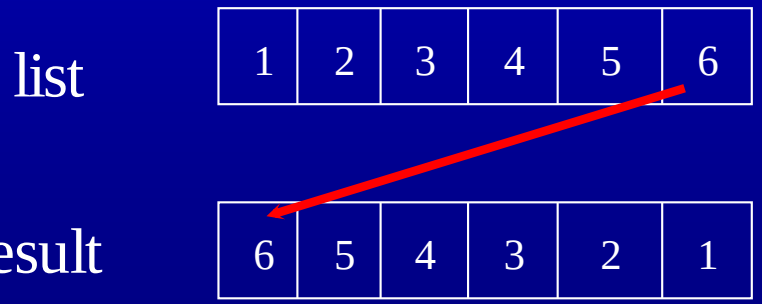
```
public static int[] reverse(int[] list) {
  int[] result = new int[list.length];

  for (int i = 0, j = result.length -
       i < list.length; i++, j--) {
    result[j] = list[i];
  }

  return result;
}
```

i = 5 ve j = 0

list[5] result[0]



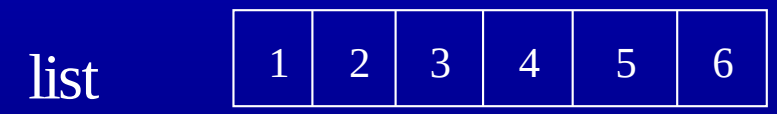
```
int[] list1 = new int[]{1, 2, 3, 4, 5, 6};
int[] list2 = reverse(list1);
```

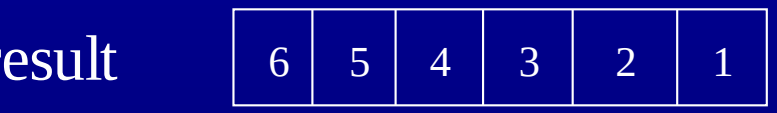
```
public static int[] reverse(int[] list) {
  int[] result = new int[list.length];

  for (int i = 0, j = result.length - 1;
       i < list.length; i++, j--) {
    result[j] = list[i];
  }

  return result;
}
```

, i 6
j -1 olur.







```
int[] list1 = new int[]{1, 2, 3, 4, 5, 6};
int[] list2 = reverse(list1);
```

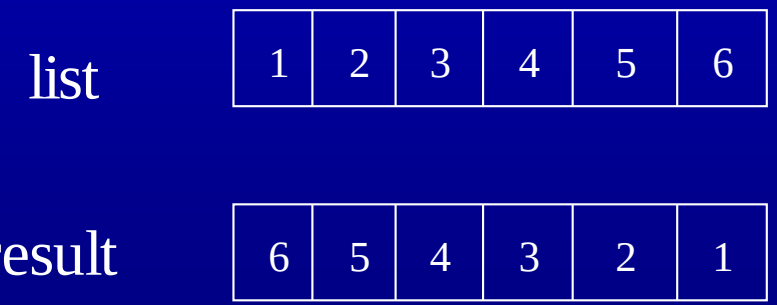
```
public static int[] reverse(int[] list) {
  int[] result = new int[list.length];

  for (int i = 0, j = result.length - 1;
       i < list.length; i++, j--) {
    result[j] = list[i];
  }

  return result;
}
```

i (=6) < 6 ifadesi false

.

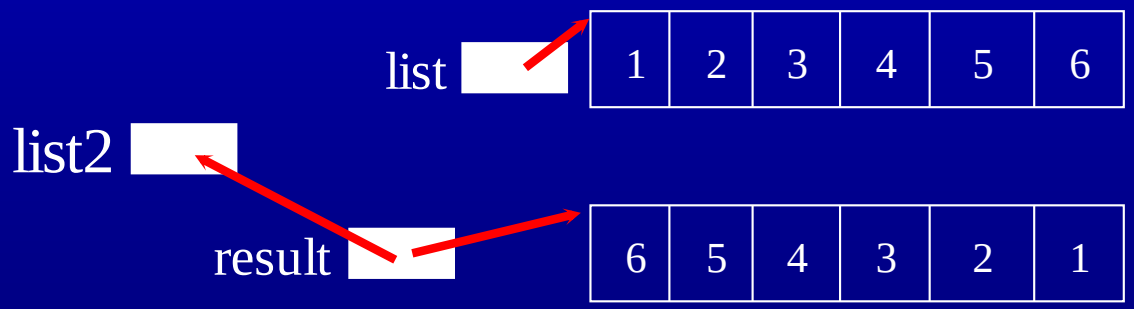


```
int[] list1 = new int[]{1, 2, 3, 4, 5, 6};
int[] list2 = reverse(list1);
```

```
public static int[] reverse(int[] list) {
  int[] result = new int[list.length];

  for (int i = 0, j = result.length - 1;
       i < list.length; i++, j--) {
    result[j] = list[i];
  }

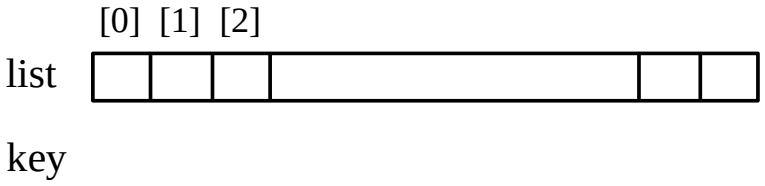
  return result;
}
```



Dizilerde Arama

Arama, bir dizideki belirli bir arama ;
belirli bir puan puan listesine dahil olup
bulmak. Arama, bilgisayar
bir . Arama
algoritma ve veri . Bu
olarak iki arama ve ikili arama
ele .

```
public class LinearSearch {
  /** The method for finding a key in the list */
  public static int linearSearch(int[] list, int key) {
    for (int i = 0; i < list.length; i++)
      if (key == list[i])
        return i;
    return -1;
  }
}
```



Linear Search)

arama anahtar
(aranan dizi listesindeki her elemanla
. Bu anahtar eleman
listedeki bir veya liste bir
bulmadan tamamlanana kadar devam
eder. arama,
anahtarla dizideki indeksini
. bulunmazsa, arama -1
.





Liang, Introduction to Java Programming, Eighth Edition, (c) 2011 Pearson Education, Inc. All rights reserved. 0132130807

```
/** The method for finding a key in the list */
public static int linearSearch(int[] list, int key) {
  for (int i = 0; i < list.length; i++)
    if (key == list[i])
      return i;
  return -1;
}
```

Metodun trace edilmesi (izlenmesi)

```
int[] list = {1, 4, 4, 2, 5, -3, 6, 2};
int i = linearSearch(list, 4);  // returns 1
int j = linearSearch(list, -4); // returns -1
int k = linearSearch(list, -3); // returns 5
```

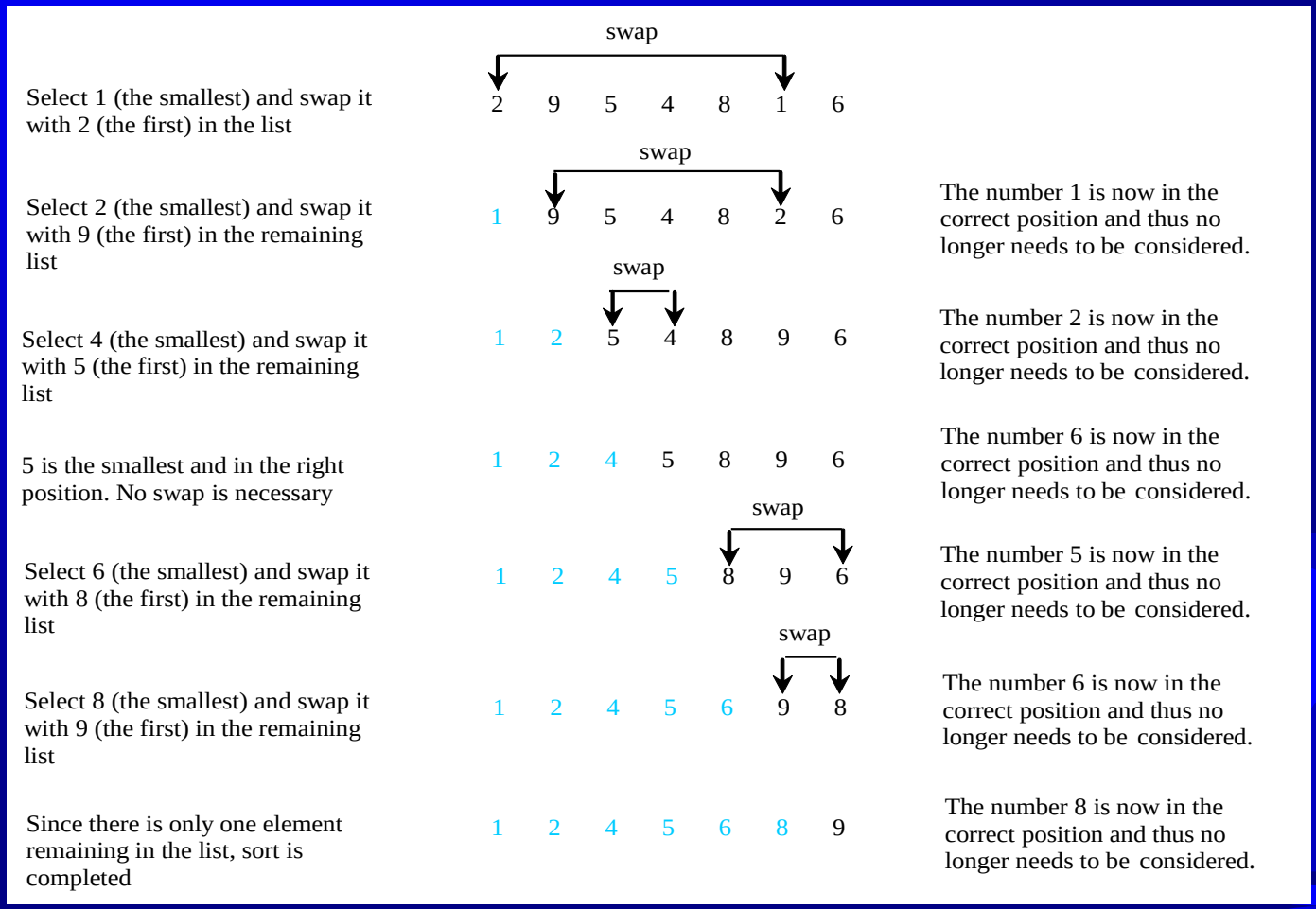
de, arama gibi bilgisayar
olarak bir
. algoritma
. Bu iki basit, sezgisel
:

(Selection sort) ve
(insertion sort).

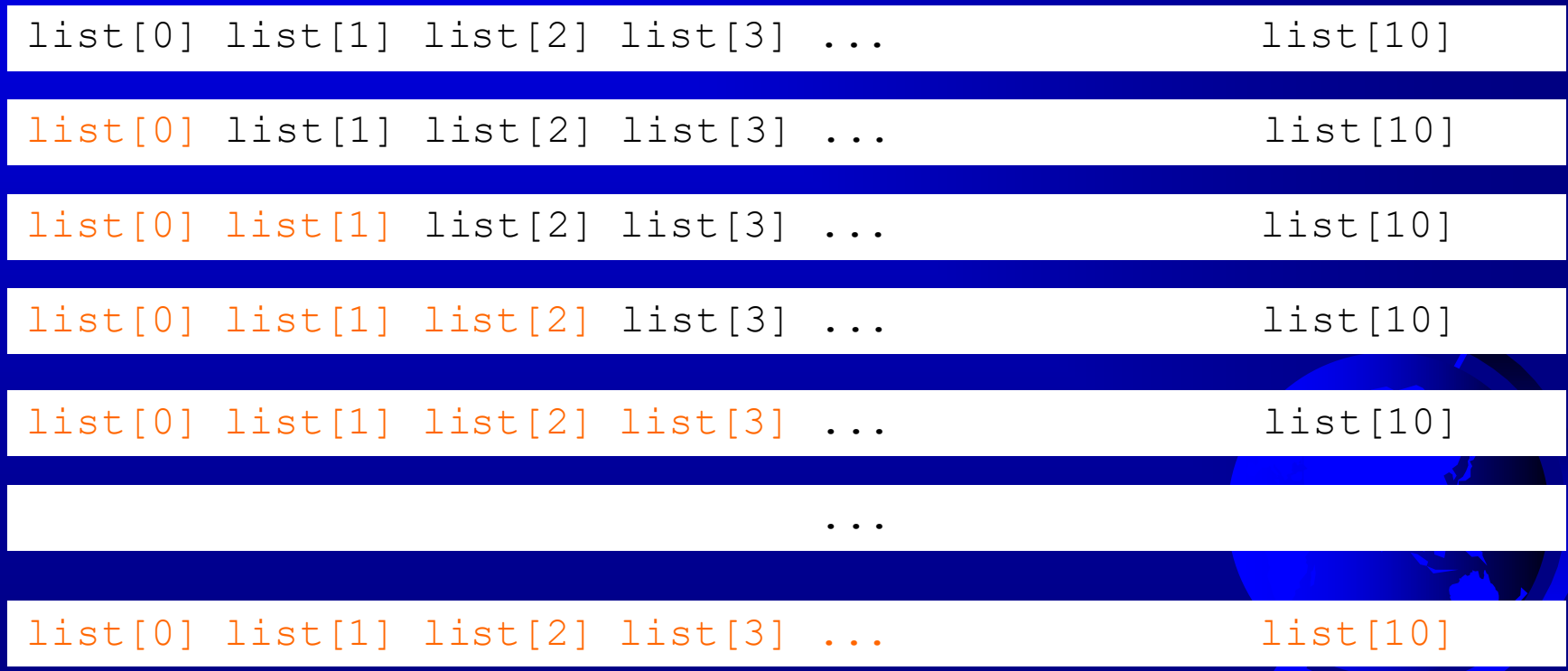


Selection Sort)

listedeki en bulur ve en sona . Daha sonra kalan
en bulur ve en sondaki ve liste bir
kadar devam eder. 6.17, kullanarak {2, 9, 5, 4, 8, 1, 6}
listesinin .



```
for (int i = 0; i < list.length; i++)
{
  select the smallest element in list[i..listSize-1];
  swap the smallest with list[i], if necessary;
  // list[i] is in its correct position.
  // The next iteration apply on list[i..listSize-1]
}
```



```
for (int i = 0; i < listSize; i++)
{
  select the smallest element in list[i..listSize-1];
  swap the smallest with list[i], if necessary;
  // list[i] is in its correct position.
  // The next iteration apply on list[i..listSize-1]
}
```

Expand

```
double currentMin = list[i];

for (int j = i; j < list.length; j++) {
  if (currentMin > list[j]) {
    currentMin = list[j];

  }
}
```



```
for (int i = 0; i < listSize; i++)
{
  select the smallest element in list[i..listSize-1];
  swap the smallest with list[i], if necessary;
  // list[i] is in its correct position.
  // The next iteration apply on list[i..listSize-1]
}
```

Expand

```
double currentMin = list[i];
int currentMinIndex = i;
for (int j = i; j < list.length; j++) {
  if (currentMin > list[j]) {
    currentMin = list[j];
    currentMinIndex = j;
  }
}
```



```
for (int i = 0; i < listSize; i++)
{
  select the smallest element in list[i..listSize-1];
  swap the smallest with list[i], if necessary;
  // list[i] is in its correct position.
  // The next iteration apply on list[i..listSize-1]
}
```

Expand

```
if (currentMinIndex != i) {
  list[currentMinIndex] = list[i];
  list[i] = currentMin;
}
```



```
/** The method for sorting the numbers */
public static void selectionSort(double[] list) {
    for (int i = 0; i < list.length; i++) {
      // Find the minimum in the list[i..list.length-1]
      double currentMin = list[i];
      int currentMinIndex = i;
      for (int j = i + 1; j < list.length; j++) {
        if (currentMin > list[j]) {
          currentMin = list[j];
          currentMinIndex = j;
        }
      }
      // Swap list[i] with list[currentMinIndex] if necessary;
      if (currentMinIndex != i) {
        list[currentMinIndex] = list[i];
        list[i] = currentMin;
      }
    }
  }
```

Invoke it

selectionSort(yourList)

rights reserved. 0132130807

The Arrays.sort Method

programlamada Java, java.util.Arrays
bir int, double, char, short, long ve float dizisini
overloaded sunar.
kod bir dizi ve bir karakter dizisini .

```
double[] numbers = {6.0, 4.4, 1.9, 2.9, 3.4, 3.5};
java.util.Arrays.sort(numbers);

char[] chars = {'a', 'A', '4', 'F', 'D', 'P'};
java.util.Arrays.sort(chars);
```

